

Stop & Wait Protocol — Quick, Structured Study Guide

1 What Is Stop & Wait?

Stop & Wait is a **flow control protocol** used at the **Data Link Layer** to prevent a sender from overwhelming a receiver.

Core Idea:

Send **one frame** → **stop** → **wait for ACK** → **send next frame**

It ensures:

- Receiver buffer does not overflow
- Synchronization between sender and receiver

2 Where It Fits in Network Layering

Layer	PDU Name	Purpose
Application	Message	User data
Transport	Segment / Datagram	End-to-end delivery
Network	Packet	Routing
Data Link	Frame	Flow + Error control
Physical	Bits	Transmission medium

Important:

At the Data Link Layer:

- Header added
- Trailer added (used for **error detection**)

3 Why Flow Control Is Needed

Speed Mismatch Example

Sender: 10 bits/sec

Receiver: 5 bits/sec

Without control:

- Receiver buffer fills
- Extra data gets lost

Flow Control Goal:

Sender transmits only when receiver is ready.

4 Versions of Stop & Wait

A) Simple Stop & Wait (Noise-less Channel)

Used when:

- No packet loss
- No corruption

Rule:

1. Send one frame
2. Wait for ACK
3. Send next frame

Problem:

If a frame or ACK is lost → **Deadlock**

- Sender waits forever
- Receiver waits forever

No error recovery mechanism.

B) Stop & Wait ARQ (Noisy Channel)

ARQ = **Automatic Repeat reQuest**

Adds **Error Control**

Three Key Mechanisms

1 Copy of Frame

Sender keeps a copy until ACK arrives.

2 Time-Out Timer

If ACK not received in time → resend frame.

3 Sequence Numbers (0 & 1)

Used to:

- Detect duplicates
- Identify retransmissions

5 Problem Handling in ARQ

Lost Data Frame

Timer expires → resend frame.

Lost ACK

Sender retransmits frame.

Receiver sees duplicate (same sequence number), discards it, and resends ACK.

Delayed ACK

Sequence number tells sender whether ACK is:

- For current frame
- Or old retransmission

6 Performance & Efficiency

Stop & Wait is inefficient because sender remains idle while waiting.

Total Time for One Frame

$$[T_{\text{total}} = T_t + 2T_p]$$

Where:

- (T_t) = Transmission delay = Frame Size / Bandwidth
- (T_p) = Propagation delay = Distance / Speed

(Processing + ACK time often ignored)

Efficiency Formula

$$[\eta = \frac{T_t}{T_t + 2T_p}]$$

If propagation delay is large $\rightarrow$ efficiency drops significantly.

Throughput

$$[\text{Throughput} = \eta \times \text{Bandwidth}]$$

OR

$$\left[\begin{array}{l} \text{Throughput} = \frac{\text{Frame Size}}{T_{\text{total}}} \end{array} \right]$$

7 Reliability Calculations

Case A: Every 4th Packet Lost

Send 10 packets
Every 4th transmission lost

Result:
13 transmissions needed to successfully send 10 packets.

Case B: Probabilistic Loss

Given:

- $n = 500$ packets
- $p = 0.2$ error probability

Formula:

$$\left[\begin{array}{l} \text{Total Transmissions} = \frac{n}{1 - p} \end{array} \right]$$

$$\begin{aligned} &= \frac{500}{0.8} \\ &= 625 \end{aligned}$$

So 625 transmissions expected to successfully deliver 500 packets.